

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Programa de Ingeniería de Sistemas y Computación
HPC

Multiplicación de matrices usando MPI

Presentado por:

Juan Esteban Cardona Blandón
Jhoan Esteban Corrales Rodríguez
Sebastian Perdomo
Juana Valentina Martínez

Profesor:

Ramiro Andrés Barrios Valencia

Pereira, Colombia

26 de mayo de 2026

Índice

1. Marco Teórico	3
1.1. Computación de Alto Rendimiento (HPC)	3
1.2. Fundamentos Matemáticos de la Multiplicación Matricial	3
1.3. Complejidad Computacional de la Multiplicación de Matrices	4
1.4. Programación Secuencial y Concurrente	4
1.5. Sistemas de Memoria Compartida	4
1.6. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria	5
1.7. Optimización Algorítmica y Aprovechamiento de la Caché	5
1.8. Métricas de Rendimiento en Computación Secuencial	6
1.9. Computación paralela	7
1.10. MPI	7
1.11. Network File System (NFS)	8
2. Marco Conceptual	9
2.1. Características de los Nodos de Ejecución	9
2.2. Características del software	10
3. Desarrollo	10
3.1. Características del software	10
3.1.1. Módulo de entrada y salida binaria	10
3.1.2. Distribución del trabajo entre procesos	10
3.1.3. Núcleo de multiplicación	11
3.1.4. Flujo de ejecución principal	11
3.1.5. Medición del tiempo de ejecución	12
3.1.6. Despliegue en el clúster AWS	12
3.2. Metodología de pruebas	12
3.2.1. Barrido de conteos de proceso	13
3.2.2. Compilación y despliegue	13
3.2.3. Generación y persistencia de matrices de entrada	13
3.2.4. Bucle de medición e idempotencia	13
3.2.5. Registro de resultados	13
3.2.6. Resumen de resultados	14
3.3. Verificación de correctitud	14
3.3.1. Verificador binario	14
3.3.2. Script de verificación	14

3.3.3.	Comprobaciones previas a la ejecución	14
3.3.4.	Reporte de resultados	15
4.	Pruebas	15
4.1.	Ejecución del algoritmo secuencial	16
4.2.	Ejecución de la implementación con MPI	17
4.2.1.	Speedup	19
4.3.	Ejecución de la implementación con MPI con optimización por compilador .	21
4.3.1.	Speedup	24
5.	Conclusiones	24

1. Marco Teórico

1.1. Computación de Alto Rendimiento (HPC)

La Computación de Alto Rendimiento (HPC, por sus siglas en inglés) consiste en la agregación de potencia de cálculo a través de arquitecturas paralelas para resolver problemas científicos y tecnológicos de extrema complejidad. Esta disciplina permite ejecutar algoritmos y modelados matemáticos que resultarían inabarcables para un sistema tradicional debido a restricciones de tiempo y recursos de hardware (Hager & Wellein, 2010a). Al dividir cargas masivas de trabajo en instrucciones más pequeñas que se procesan simultáneamente, HPC maximiza la velocidad de procesamiento de datos y la obtención de resultados en la ingeniería (Patterson & Hennessy, 2017a).

1.2. Fundamentos Matemáticos de la Multiplicación Matricial

La multiplicación de matrices es una operación algebraica bidimensional fundamental que procesa dos matrices de origen para producir una tercera. Dadas dos matrices A de dimensión $m \times p$ y B de dimensión $p \times n$, su producto da como resultado una matriz C de dimensiones $m \times n$. Matemáticamente, el valor de cada celda individual C_{ij} en la matriz resultante se calcula mediante el producto punto o escalar entre la i -ésima fila de la matriz A y la j -ésima columna de la matriz B . Esta operación se expresa de forma analítica mediante la siguiente sumatoria:

$$C_{ij} = \sum_{k=1}^p A_{ik} B_{kj}$$

Para que esta operación esté matemáticamente definida y sea computacionalmente viable, existe una condición estricta de dimensionalidad: el número de columnas de la primera matriz (p) debe coincidir exactamente con el número de filas de la segunda matriz (Strang, 2016).

En el contexto de la ingeniería y la Computación de Alto Rendimiento (HPC), la multiplicación matricial trasciende la teoría pura para consolidarse como la rutina principal (*kernel*) subyacente en la gran mayoría del software científico. Esta operación constituye el pilar del Nivel 3 de la especificación BLAS (*Basic Linear Algebra Subprograms*), el estándar de facto para operaciones de álgebra lineal. La eficiencia con la que un procesador ejecuta este algoritmo dicta directamente el rendimiento de aplicaciones críticas modernas, tales como el modelado de dinámica de fluidos, la física de partículas, el procesamiento masivo de imágenes y, muy especialmente, el entrenamiento de arquitecturas profundas en inteligencia artificial mediante redes neuronales (Dongarra et al., 1990).

1.3. Complejidad Computacional de la Multiplicación de Matrices

La multiplicación de matrices es un problema canónico en el álgebra lineal computacional, utilizado frecuentemente para evaluar el desempeño y la robustez de las arquitecturas de hardware. Dadas dos matrices cuadradas A y B de dimensiones $n \times n$, el algoritmo iterativo secuencial clásico exige iterar mediante tres ciclos anidados, realizando matemáticamente n^3 multiplicaciones y $n^2(n - 1)$ sumas (Cormen et al., 2009).

Esto establece una complejidad temporal asintótica estricta de $\mathcal{O}(n^3)$. Este crecimiento cúbico implica que, si el tamaño de la matriz se duplica, el esfuerzo computacional se multiplica por un factor de ocho. Debido a esta pronunciada curva de costo algorítmico, la distribución de la carga mediante técnicas de programación paralela (hilos o procesos) se vuelve un requisito indispensable cuando se operan matrices de dimensiones elevadas (Grama et al., 2003a).

1.4. Programación Secuencial y Concurrente

La programación secuencial es el paradigma clásico de desarrollo donde las instrucciones de un algoritmo se ejecutan de forma estrictamente lineal, una tras otra, sobre un único núcleo de procesamiento. En este modelo tradicional, el límite de rendimiento está directamente dictado por la frecuencia de reloj del hardware. Como respuesta a estas limitaciones físicas, la programación concurrente estructura el software en múltiples hilos o tareas independientes que hacen progreso de forma superpuesta en el tiempo. Cuando estas tareas operan de manera simultánea en una arquitectura de hardware multinúcleo, la concurrencia se traduce en paralelismo físico, lo cual permite dividir la carga de trabajo y reducir drásticamente el tiempo de ejecución en problemas de cómputo intensivo (Ben-Ari, 2006).

1.5. Sistemas de Memoria Compartida

En el ámbito de la computación paralela, el modelo de memoria compartida establece que múltiples hilos de ejecución (como los creados por la librería Pthreads) operan dentro de un mismo espacio de direccionamiento lógico. A nivel de arquitectura, esto significa que todos los núcleos del procesador pueden acceder de forma directa y simultánea a las mismas estructuras de datos residentes en la memoria principal. Para problemas matriciales, este modelo resulta excepcionalmente eficiente frente a los sistemas de memoria distribuida, ya que evita la necesidad de copiar y transmitir los datos de las matrices de origen repetidamente entre procesos. Aunque este paradigma exige control riguroso para evitar condiciones de carrera, en operaciones intrínsecamente independientes como el cálculo individual de las celdas de una

matriz resultado, su implementación maximiza la eficiencia computacional sin sobrecarga de sincronización (Silberschatz et al., 2018).

1.6. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria

El rendimiento de una aplicación de alto rendimiento (HPC) no depende exclusivamente de la capacidad matemática del procesador, sino también del acceso al subsistema de memoria. Para mitigar la alta latencia térmica y física de la memoria RAM, los procesadores modernos integran una jerarquía de memoria ultrarrápida cercana a los núcleos, conocida como memoria Caché, habitualmente dividida en niveles (L1, L2 y L3). La unidad mínima de transferencia de información entre la RAM principal y la memoria caché no es un byte aislado, sino un bloque continuo de datos denominado línea de caché (*cache line*), que en las arquitecturas modernas suele constar de 64 bytes (Drepper, 2007).

Esta transferencia por bloques está fundamentada en el principio de localidad espacial de memoria, un concepto empírico que dicta que si un programa accede a una dirección de memoria, es inminentemente probable que acceda a las direcciones adyacentes a continuación. En lenguajes de programación como C, las matrices bidimensionales se almacenan en la memoria RAM de manera contigua organizadas por filas (*row-major order*). Durante la multiplicación matricial clásica iterativa, la segunda matriz se recorre inevitablemente por columnas; este patrón de acceso ortogonal rompe por completo la localidad espacial. Al intentar leer datos saltando grandes bloques de memoria, los valores necesarios no se encuentran en la línea de caché cargada, provocando fallos de caché (*cache misses*) que obligan al procesador a detenerse (*stall*) mientras busca los datos nuevamente en la lenta memoria RAM, lo cual penaliza severamente el desempeño del algoritmo (Patterson & Hennessy, 2017a).

1.7. Optimización Algorítmica y Aprovechamiento de la Caché

Aunque la paralelización distribuye la carga de trabajo, el cuello de botella de la memoria solo puede resolverse reestructurando el algoritmo subyacente. La estrategia de optimización más directa para la multiplicación de matrices en C consiste en el intercambio de bucles (*loop interchange*). Altera el orden tradicional de los ciclos anidados de i, j, k a i, k, j . Matemáticamente, el resultado es idéntico, pero a nivel de hardware, el bucle más interno ahora recorre las matrices B y C estrictamente por filas. Esto garantiza que el procesador consuma por completo los 64 bytes de cada línea de caché (*cache line*) cargada antes de solicitar la siguiente, aprovechando al máximo la localidad espacial y reduciendo las penalizaciones por fallos de caché en órdenes de magnitud (Lam et al., 1991).

Para matrices masivas que exceden la capacidad total de la memoria caché (L1 y L2),

la literatura de HPC emplea una técnica avanzada denominada multiplicación por bloques o *Loop Tiling*. Este método particiona las matrices originales en submatrices (o bloques) de un tamaño cuidadosamente calculado para que residan por completo dentro de la caché rápida del procesador. En lugar de multiplicar filas y columnas completas, el algoritmo opera sobre estos bloques aislados, reutilizando los datos cargados repetidas veces antes de que el controlador de memoria los descarte (*eviction*). Esta técnica transforma un problema limitado por el ancho de banda de la memoria (*memory-bound*) en uno puramente computacional (*compute-bound*) (Grama et al., 2003a).

1.8. Métricas de Rendimiento en Computación Secuencial

El análisis de un algoritmo secuencial requiere establecer una línea base absoluta sobre la cual evaluar cualquier mejora algorítmica o de hardware. La métrica fundamental empírica es el Tiempo de Ejecución (*Wall-clock time*), que representa el tiempo físico real transcurrido desde el inicio hasta la finalización del programa. Sin embargo, dado que este valor fluctúa dependiendo de la interferencia del sistema operativo y los accesos a memoria, la arquitectura de computadores recurre a métricas de rendimiento intrínseco (*throughput*) (Patterson & Hennessy, 2017a).

Para cargas de trabajo de alto rendimiento algebraico como la multiplicación de matrices, la métrica estandarizada predominante es el volumen de Operaciones de Punto Flotante por Segundo (FLOPS, por sus siglas en inglés). Esta medida cuantifica el rendimiento matemático bruto del procesador y se define mediante la siguiente relación:

$$\text{FLOPS} = \frac{\text{Operaciones Aritméticas Totales}}{T_{\text{ejecución}}} \quad (1)$$

Para el algoritmo clásico iterativo que multiplica dos matrices cuadradas de dimensión $n \times n$, el número total de operaciones de punto flotante se establece analíticamente en $2n^3$ (contabilizando una instrucción de multiplicación y una de suma por cada iteración del bucle más interno). Al contrastar los FLOPS experimentales obtenidos en la ejecución secuencial contra el rendimiento teórico máximo (*Peak FLOPS*) que el fabricante del procesador garantiza para un solo núcleo, es posible diagnosticar ineficiencias de memoria o cuellos de botella antes de recurrir a la paralelización (Hager & Wellein, 2010a).

Adicionalmente, el rendimiento de la ejecución en un único núcleo se evalúa a nivel microarquitectónico mediante el IPC (Instrucciones por Ciclo de reloj). Esta métrica indica la eficiencia con la que el código compilado logra mantener ocupada la segmentación (*pipeline*) del procesador, minimizando las detenciones o burbujas producidas por dependencias de datos en la ejecución lineal (Yi et al., 2020).

1.9. Computación paralela

La computación paralela es un paradigma de procesamiento en el que un problema de gran tamaño se divide en subproblemas más pequeños que pueden resolverse de forma simultánea mediante múltiples procesadores o núcleos de ejecución. Su objetivo principal es reducir el tiempo total de cómputo y aumentar la eficiencia de uso de los recursos disponibles, especialmente en problemas con alta carga aritmética o grandes volúmenes de datos (IBM, 2024).

Desde una perspectiva conceptual, la computación paralela contrasta con la computación secuencial, en la cual las instrucciones se ejecutan una tras otra sobre un único procesador. En entornos paralelos, el rendimiento no depende únicamente de la capacidad de cálculo del procesador, sino también de la forma en que se particiona el trabajo, se distribuyen los datos y se coordinan los procesos durante la ejecución (Grama et al., 2003b).

En la práctica, la computación paralela puede implementarse sobre diferentes arquitecturas, entre ellas memoria compartida, memoria distribuida y esquemas híbridos. En sistemas de memoria distribuida, cada proceso posee su propia memoria local y la coordinación entre nodos se realiza mediante intercambio explícito de mensajes. Este modelo resulta especialmente útil en clústeres y supercomputadores, donde la escalabilidad depende de minimizar la sobrecarga de comunicación y maximizar la proporción entre cómputo útil y costo de coordinación (Hager & Wellein, 2010b).

La relevancia de la computación paralela en computación de alto rendimiento proviene de su capacidad para resolver problemas que serían impracticables en forma secuencial. Aplicaciones como simulación numérica, álgebra lineal densa, análisis de datos y aprendizaje automático se benefician de este paradigma debido a que presentan un alto grado de paralelismo natural y un costo computacional considerable (Patterson & Hennessy, 2017b).

1.10. MPI

MPI (*Message Passing Interface*) es un estándar para programación paralela mediante paso de mensajes, diseñado principalmente para sistemas de memoria distribuida. Su propósito es ofrecer una interfaz portable, eficiente y ampliamente soportada para desarrollar aplicaciones paralelas en lenguajes como C, C++ y Fortran (MPI Forum, 2021; Snir et al., 1998).

Una de las características fundamentales de MPI es que el programador controla explícitamente la descomposición del problema, la distribución de datos y la comunicación entre procesos. Esto permite construir aplicaciones con un modelo SPMD (*Single Program, Multiple Data*), en el cual múltiples procesos ejecutan el mismo programa pero trabajan sobre

datos distintos (Gropp et al., 2014).

En MPI, los procesos se organizan dentro de comunicadores, que definen conjuntos de procesos autorizados para intercambiar mensajes. Cada proceso dentro de un comunicador posee un rango (*rank*) que lo identifica de forma única y permite establecer el origen o destino de la comunicación. Esta abstracción facilita tanto las comunicaciones punto a punto como las operaciones colectivas.

Entre las operaciones más utilizadas de MPI se encuentran el envío y recepción de mensajes entre procesos, así como las comunicaciones colectivas como difusión (*broadcast*), dispersión (*scatter*), reunión (*gather*) y reducción (*reduce*). Estas primitivas permiten implementar de manera estructurada algoritmos paralelos donde los datos deben repartirse, procesarse localmente y recombinarse al final de la ejecución (MPI Forum, 2021).

En el contexto de la multiplicación de matrices, MPI es particularmente adecuado porque el problema puede descomponerse por filas o bloques, de manera que cada proceso calcula una parte independiente de la matriz resultado. No obstante, el rendimiento final depende de la relación entre el tamaño del problema, el costo de comunicación y la eficiencia del núcleo de cómputo local. Por esta razón, las implementaciones MPI suelen requerir además optimización algorítmica y cuidadosa atención al acceso a memoria (Pacheco, 1997; Snir et al., 1998).

1.11. Network File System (NFS)

El Network File System (NFS) es un protocolo de sistema de archivos distribuido que permite a varios nodos acceder a archivos almacenados de forma remota como si estuvieran disponibles localmente en el sistema operativo. Su principal ventaja es que proporciona un mecanismo transparente de compartición de datos, de manera que los procesos pueden leer y escribir archivos ubicados en un servidor central sin requerir una lógica explícita de transferencia de archivos en la aplicación (Sandberg et al., 1985; Stoner, Terry et al., 1995).

En entornos de computación paralela y distribuida, NFS resulta útil para compartir binarios, datos de entrada y resultados entre múltiples nodos de un clúster. Esta característica simplifica el despliegue de aplicaciones MPI, ya que todos los procesos pueden acceder a la misma estructura de directorios y ejecutar el mismo binario sin necesidad de copiar manualmente los archivos en cada máquina (Storm et al., 1988; Tanenbaum & Steen, 2015).

Una propiedad importante de NFS es que el acceso a archivos remotos introduce costos adicionales de red y de sincronización frente a un sistema de archivos local. Aunque el cliente percibe la operación como una lectura o escritura convencional, la solicitud debe transmitirse al servidor y esperar respuesta, lo que puede afectar el tiempo total de ejecución si el volumen de accesos es alto o si el sistema se utiliza simultáneamente por varios procesos (Stone &

Hennessy, 1995; Tanenbaum & Steen, 2015).

En un clúster MPI, este comportamiento tiene implicaciones directas sobre el rendimiento. Si los archivos de entrada se leen desde NFS al inicio de la ejecución, la sobrecarga puede ser aceptable; sin embargo, si la aplicación depende de accesos frecuentes o si varios procesos realizan lecturas concurrentes sobre archivos grandes, la latencia de red y el ancho de banda del sistema de almacenamiento pueden convertirse en un cuello de botella. Por esta razón, el uso de NFS debe analizarse no solo como una comodidad operativa, sino también como un factor que influye en las mediciones experimentales (Stone & Hennessy, 1995; Terry, 1995).

En este proyecto, NFS cumple una función de infraestructura para distribuir los binarios, leer las matrices de entrada y almacenar resultados en una ubicación común. Esto facilita la ejecución homogénea en todos los nodos y evita inconsistencias entre entornos locales, pero al mismo tiempo introduce una capa adicional de dependencia sobre el subsistema de almacenamiento compartido, que debe considerarse al interpretar los tiempos medidos (Sandberg et al., 1985; Tanenbaum & Steen, 2015).

2. Marco Conceptual

2.1. Características de los Nodos de Ejecución

Para este proyecto se usaron para los nodos (Head Node y Compute Node) instancias de AWS EC2 m7i-flex.large, las cuales cuentan con las siguientes características:

- Instancia utilizada: AWS EC2 m7i-flex.large
- Procesador: Intel Xeon Scalable (Sapphire Rapids, 4ta generación)
- Cantidad de vCPUs: 2
- Cantidad de núcleos: 1
- Cantidad de subprocesos/hilos: 2
- Frecuencia del procesador: hasta 3.2 GHz
- Memoria RAM: 8 GiB DDR5
- Arquitectura: x86_64 (64 bits)
- Almacenamiento: EBS Only
- Ancho de banda de red: hasta 12.5 Gbps
- Sistema operativo: Linux

2.2. Características del software

Se puede encontrar el código fuente del proyecto en el siguiente repositorio de GitHub:
<https://github.com/Juanes7222/HPC-Matrix.git>

3. Desarrollo

A continuación se presenta el desarrollo de la solución, que incluye tanto la implementación de la multiplicación de matrices con MPI como la forma en que se realizaron las pruebas.

3.1. Características del software

El programa realiza la multiplicación de matrices cuadradas de orden n y divide el trabajo entre varios procesos MPI. Para ello, reparte las filas de la matriz A entre los procesos disponibles, de manera que cada uno calcule una parte del resultado final. La matriz B , en cambio, no se divide entre procesos: cada uno la lee desde un sistema de archivos compartido NFS y la mantiene en memoria local durante el cálculo.

La solución se compone de cuatro partes principales: el módulo encargado de leer y escribir matrices en formato binario, la función que organiza cómo se reparte el trabajo, el núcleo que hace la multiplicación y la función principal que coordina todo el proceso MPI.

3.1.1. Módulo de entrada y salida binaria

El módulo `matrix_io` define el formato de archivo usado por el programa. Cada archivo guarda primero el tamaño de la matriz y luego todos sus valores en orden por filas. Esto permite leer y escribir las matrices de forma sencilla y rápida.

La función `matrix_read_bin` abre el archivo, lee su tamaño, reserva memoria y carga todos los datos de una sola vez. Si ocurre algún problema, como que el archivo no exista o esté incompleto, la función devuelve error. Por su parte, `matrix_write_bin` hace el proceso inverso: guarda primero la dimensión de la matriz y luego sus valores.

Usar un bloque continuo de memoria facilita tanto la lectura y escritura como el trabajo de MPI, porque los datos pueden enviarse y recibirse sin realizar copias intermedias innecesarias.

3.1.2. Distribución del trabajo entre procesos

La función `build_distribution` se encarga de repartir las filas de la matriz A entre los procesos disponibles. La idea es simple: cada proceso recibe una cantidad parecida de filas, y si sobran algunas, se reparten entre los primeros procesos.

Con esta información se construyen dos arreglos necesarios para las operaciones colectivas de MPI: uno indica cuántos datos recibe cada proceso y el otro indica desde qué posición del arreglo general comienza cada bloque. Estos arreglos permiten usar `MPI_Scatterv` para repartir la matriz A y `MPI_Gatherv` para reunir el resultado final.

3.1.3. Núcleo de multiplicación

El cálculo local reutiliza la función `matrixMultiplyRangeFlat`, que también se usa en la versión secuencial. Esto permite comparar ambas implementaciones sobre el mismo núcleo de cálculo.

Cada proceso multiplica sus filas asignadas de A por la matriz completa B . Para mejorar el rendimiento, el orden de los bucles se organiza de forma que el acceso a memoria sea más eficiente, especialmente en la matriz B . Esto ayuda a aprovechar mejor la caché y reduce tiempos de espera cuando las matrices son grandes.

3.1.4. Flujo de ejecución principal

La función principal controla todo el ciclo de vida de MPI. Primero inicializa el entorno, obtiene el rango de cada proceso y verifica que los argumentos de ejecución sean correctos. Si algo está mal, el programa termina de forma ordenada.

La ejecución se divide en cinco etapas: lectura, distribución, cómputo, recolección y escritura final. Cada etapa se mide por separado para poder analizar con claridad dónde se invierte el tiempo.

Fase de E/S. El proceso principal lee la matriz A desde disco y comparte su tamaño con los demás procesos. Después, todos los procesos leen por su cuenta la matriz B desde NFS. Con esto se evita tener que enviar B completa por MPI, lo que reduce comunicación entre procesos. Cada proceso verifica además que ambas matrices tengan la misma dimensión.

Fase de distribución. Una vez cargadas las matrices, el proceso principal reparte las filas de A entre los procesos mediante `MPI_Scatterv`. Cada uno recibe solo la parte que le corresponde y libera la matriz completa cuando ya no la necesita.

Fase de cómputo. Cada proceso realiza la multiplicación de su bloque local de filas por la matriz B . Como cada uno trabaja sobre una parte independiente del problema, no es necesaria comunicación adicional durante este paso.

Fase de recolección. Cuando todos terminan, el proceso principal reúne los bloques parciales del resultado usando `MPI_Gatherv`. Esta operación es necesaria porque no todos los procesos reciben la misma cantidad de filas cuando n no se divide exactamente entre el número de procesos.

Fase de escritura. Finalmente, solo el proceso principal guarda el resultado final en disco. El tiempo total reportado incluye también esta última escritura, de manera que la medición refleje el costo completo de la ejecución.

3.1.5. Medición del tiempo de ejecución

Para medir el rendimiento, el programa registra el tiempo de cada etapa usando `MPI_Wtime`. Luego, los tiempos de todos los procesos se reducen al valor máximo, porque en una ejecución paralela el tiempo real observado es el del proceso que tarda más.

El programa reporta así cinco valores: tiempo de lectura, tiempo de distribución, tiempo de cómputo, tiempo de recolección y tiempo total. Estos valores se imprimen en un formato que luego puede ser procesado automáticamente por el script de pruebas.

3.1.6. Despliegue en el clúster AWS

Las pruebas se ejecutaron en un clúster de tres instancias EC2 `m7i-flex.large`, cada una con 2 vCPUs y 8 GiB de RAM. Todas las máquinas están conectadas dentro de la misma red privada de AWS, lo que permite ejecutar MPI sin salir de la subred.

El nodo principal exporta un directorio compartido mediante NFS, y ese mismo directorio es montado por todos los nodos. Allí se almacenan los binarios, las matrices de entrada, los resultados y los archivos temporales. Esto asegura que todos los procesos ejecuten exactamente los mismos archivos.

La compilación se realiza con `mpicc` usando GCC y Open MPI. Además, el archivo `hostfile` define cuántos procesos puede ejecutar cada nodo, permitiendo controlar la distribución durante las pruebas.

3.2. Metodología de pruebas

Las pruebas de rendimiento se automatizan con el script `bench_mpi_nfs.sh`. Este script se encarga de compilar, desplegar binarios, generar matrices de prueba, ejecutar distintas configuraciones y guardar los resultados en un CSV.

El script permite elegir entre una versión normal y una versión optimizada del programa. La versión optimizada usa banderas de compilación más agresivas, mientras que la versión

normal se compila con opciones básicas de advertencia.

3.2.1. Barrido de conteos de proceso

Cuando no se indican manualmente los procesos a probar, el script toma la información del `hostfile` y genera automáticamente un conjunto de ejecuciones desde 1 hasta el máximo disponible. En este caso, con tres nodos, el barrido cubre las configuraciones de 1, 2 y 3 procesos.

3.2.2. Compilación y despliegue

Antes de cada serie de pruebas, el script verifica si el binario ya está compilado y actualizado. Si no lo está, lo recompila y lo copia al directorio compartido para que todos los nodos usen la misma versión. Esto evita inconsistencias entre ejecuciones.

3.2.3. Generación y persistencia de matrices de entrada

Para cada tamaño de matriz, el script verifica si ya existen los archivos de entrada. Si no existen, los genera una sola vez y luego los reutiliza. Esto hace que todas las ejecuciones se comparen sobre exactamente los mismos datos y evita que la lectura se vea afectada por la generación reciente de archivos.

3.2.4. Bucle de medición e idempotencia

El bucle principal recorre las distintas combinaciones de tamaño, número de procesos y réplicas. Antes de ejecutar un caso, el script revisa si ese resultado ya está en el CSV. Si ya existe, lo omite. Gracias a eso, el benchmark puede reanudarse sin repetir datos.

Cada ejecución se lanza con `mpirun`, y si ocurre un error, el script registra el fallo y continúa con el siguiente caso. De esta manera, una prueba problemática no interrumpe toda la campaña de medición.

3.2.5. Registro de resultados

Cada medición se guarda inmediatamente en el CSV. El archivo incluye información sobre la máquina, la versión compilada, el número de procesos, el tamaño de la matriz, la réplica y los tiempos de cada fase.

Todos los tiempos reportados corresponden al proceso más lento, porque ese es el valor que mejor representa el costo real de una ejecución paralela sincronizada.

3.2.6. Resumen de resultados

Al finalizar las pruebas, el script calcula promedios de tiempo y speedup para facilitar el análisis. De esta forma es posible comparar el comportamiento de la versión optimizada frente a la no optimizada, tanto en el cómputo puro como en el tiempo total de ejecución.

3.3. Verificación de correctitud

Para comprobar que el resultado producido por la implementación paralela es correcto, se utiliza un verificador independiente junto con un script que organiza las pruebas.

3.3.1. Verificador binario

El programa `verify_mpi` actúa como referencia secuencial. Recibe como entrada las matrices A , B y el resultado C , vuelve a calcular la multiplicación y compara ambos resultados elemento por elemento.

Si todo coincide, el programa imprime un mensaje de éxito. Si encuentra diferencias, informa cuántos elementos son incorrectos y muestra algunos ejemplos. Como los datos son enteros, la comparación es exacta.

3.3.2. Script de verificación

El script `verify_mpi_nfs.sh` ejecuta varios casos pensados para cubrir situaciones normales y casos límite. Entre ellos hay pruebas con un solo proceso, con distribución uniforme, con distribución irregular y con más procesos que filas.

Estas pruebas son importantes porque permiten revisar los puntos donde es más fácil cometer errores: el reparto de filas, el cálculo de desplazamientos y el manejo de procesos que no reciben datos.

3.3.3. Comprobaciones previas a la ejecución

Antes de iniciar las pruebas, el script verifica que el directorio compartido esté disponible, que se puedan escribir archivos de prueba y que `mpirun` funcione correctamente. Si alguna de estas comprobaciones falla, el proceso se detiene de inmediato para evitar resultados inválidos.

Las matrices de entrada se generan una sola vez y se reutilizan en todas las pruebas del mismo tamaño. Al terminar, el directorio temporal se elimina automáticamente para no dejar archivos residuales.

3.3.4. Reporte de resultados

Cada caso de prueba se reporta como **PASS**, **FAIL** o **SKIP**, según el resultado. Al final se imprime un resumen general con el número total de casos exitosos, fallidos y omitidos.

El script termina con código de salida cero solo si no hubo fallos, por lo que también puede integrarse fácilmente en un flujo de integración continua.

4. Pruebas

En esta sección se presentan las pruebas realizadas para validar el funcionamiento del sistema desarrollado. Se describen los casos de prueba, los resultados obtenidos y las conclusiones derivadas de dichas pruebas. Se hace la comparación entre los resultados de las dos máquinas descritas anteriormente, con el fin de evaluar el rendimiento del sistema en diferentes entornos.

Para las optimizaciones por compilador se usaron las banderas `-O3 -Wall -march=native -funroll-loops -flto -ffast-math -fomit-frame-pointer`.

El significado de cada bandera es el siguiente:

- **-O3:** Esta bandera habilita un nivel de optimización agresivo, permitiendo al compilador aplicar una amplia gama de técnicas para mejorar el rendimiento del código.
- **-Wall:** Activa la mayoría de las advertencias del compilador, lo que ayuda a identificar posibles problemas en el código que podrían afectar su rendimiento o funcionalidad.
- **-march=native:** Permite al compilador generar código optimizado para la arquitectura específica del procesador en el que se está compilando, aprovechando las características y capacidades de hardware disponibles.
- **-funroll-loops:** Habilita el desenrollado de bucles, lo que puede mejorar el rendimiento al reducir la sobrecarga de control de bucle y aumentar la paralelización de las instrucciones.
- **-flto:** Habilita la optimización a nivel de enlace, lo que permite al compilador realizar optimizaciones globales en todo el programa, incluso entre diferentes archivos de código fuente.
- **-ffast-math:** Permite al compilador realizar optimizaciones agresivas en operaciones matemáticas, lo que puede mejorar el rendimiento a costa de la precisión en algunos casos como la evaluación de funciones trigonométricas.

- **-fomit-frame-pointer:** Indica al compilador que no utilice un puntero de marco para las funciones, es decir, que no cree un marco de pila para cada función, lo que puede mejorar el rendimiento al reducir la cantidad de instrucciones necesarias para gestionar la pila de llamadas.

4.1. Ejecución del algoritmo secuencial

Se hizo la ejecución del algoritmo secuencial en una de las maquinas `m7i-flex.large` para mantener la validez del experimento. Para la ejecución de este algoritmo se hizo la compilación usando las flags antes mencionadas.

Table 1 — Individual Measurements (ms)					
Sequential — with -O3 (reference baseline)					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	43,334	519,163	5.138,222	122.490,019	1.223.039,228
2	43,432	522,379	5.084,896	121.860,622	1.225.812,911
3	41,498	520,105	5.123,672	121.319,806	1.220.988,627
4	42,278	510,917	5.007,324	120.409,703	1.226.275,340
5	43,303	525,133	5.108,166	121.733,234	1.223.618,296
6	42,969	518,765	5.038,193	120.965,854	1.231.812,568
7	42,106	514,992	5.000,949	118.512,597	1.220.079,626
8	43,507	526,472	5.065,711	121.165,981	1.224.319,297
9	41,716	517,814	5.034,278	120.653,970	1.228.423,724
10	44,418	538,820	5.305,733	123.468,355	1.226.642,785
Average	42,856	521,456	5.090,714	121.258,014	1.225.101,240
Std Dev	0,876	7,221	84,555	1.253,470	3.309,604
CV (%)	2,04	1,38	1,66	1,03	0,27

Cuadro 1: Tiempo de ejecución del algoritmo secuencial

4.2. Ejecución de la implementación con MPI

MPI 2 process(es) — no compiler optimization					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	179,561	1.361,672	11.005,996	86.509,157	691.507,332
2	179,539	1.379,274	11.040,429	86.909,363	647.003,325
3	169,456	1.355,740	10.904,573	86.989,003	609.102,433
4	172,734	1.360,467	10.945,073	86.471,017	686.812,137
5	171,582	1.365,557	10.996,141	86.986,724	687.594,459
6	174,913	1.381,816	10.816,048	86.115,907	688.673,028
7	174,926	1.376,927	10.956,205	86.772,250	642.659,254
8	170,003	1.356,712	10.845,720	86.506,994	585.276,191
9	173,819	1.363,879	10.991,310	86.535,568	687.684,328
10	87,262	1.367,905	11.033,276	85.995,063	689.784,018
Average	165,379	1.366,995	10.953,477	86.579,105	661.609,650
Std Dev	26,245	8,860	72,737	325,557	36.832,899
CV (%)	15,87	0,65	0,66	0,38	5,57

Cuadro 2: Tiempo de ejecución de la implementación con MPI con 2 procesos

MPI 3 process(es) — no compiler optimization					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	151,219	958,840	7.415,483	58.725,009	404.834,814
2	146,528	945,306	7.309,129	57.987,980	421.157,472
3	146,682	941,824	7.262,886	57.640,459	425.329,073
4	142,542	956,876	7.342,657	57.833,956	459.535,078
5	143,413	940,526	7.255,265	57.728,686	426.325,923
6	144,629	948,021	7.380,258	58.354,026	462.283,402
7	147,717	967,417	7.421,732	57.820,588	401.512,770
8	148,638	949,110	7.349,965	57.999,970	417.570,723
9	144,969	945,662	7.340,478	46.388,965	460.545,119
10	146,068	948,126	7.257,170	57.954,298	459.871,075
Average	146,240	950,171	7.333,502	56.843,394	433.896,545
Std Dev	2,427	7,963	58,842	3.497,922	23.023,458
CV (%)	1,66	0,84	0,80	6,15	5,31

Cuadro 3: Tiempo de ejecución de la implementación con MPI con 3 procesos

MPI 4 process(es) — no compiler optimization					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	118,042	757,604	5.809,780	45.157,294	344.542,689
2	118,284	763,654	5.824,567	45.881,496	357.989,764
3	118,209	762,779	5.721,462	45.916,569	362.022,222
4	118,341	758,361	5.828,125	45.392,908	359.592,785
5	119,297	760,729	5.846,010	45.908,716	350.498,743
6	118,445	764,383	5.839,322	45.912,984	362.986,961
7	118,036	765,937	5.736,196	45.931,691	359.207,728
8	118,344	761,433	5.753,193	45.897,086	356.656,881
9	118,123	762,713	5.783,572	44.724,787	362.755,429
10	118,071	759,836	5.828,061	46.284,776	362.947,006
Average	118,319	761,743	5.797,029	45.700,831	357.920,021
Std Dev	0,353	2,521	43,073	440,809	5.755,335
CV (%)	0,30	0,33	0,74	0,96	1,61

Cuadro 4: Tiempo de ejecución de la implementación con MPI con 4 procesos

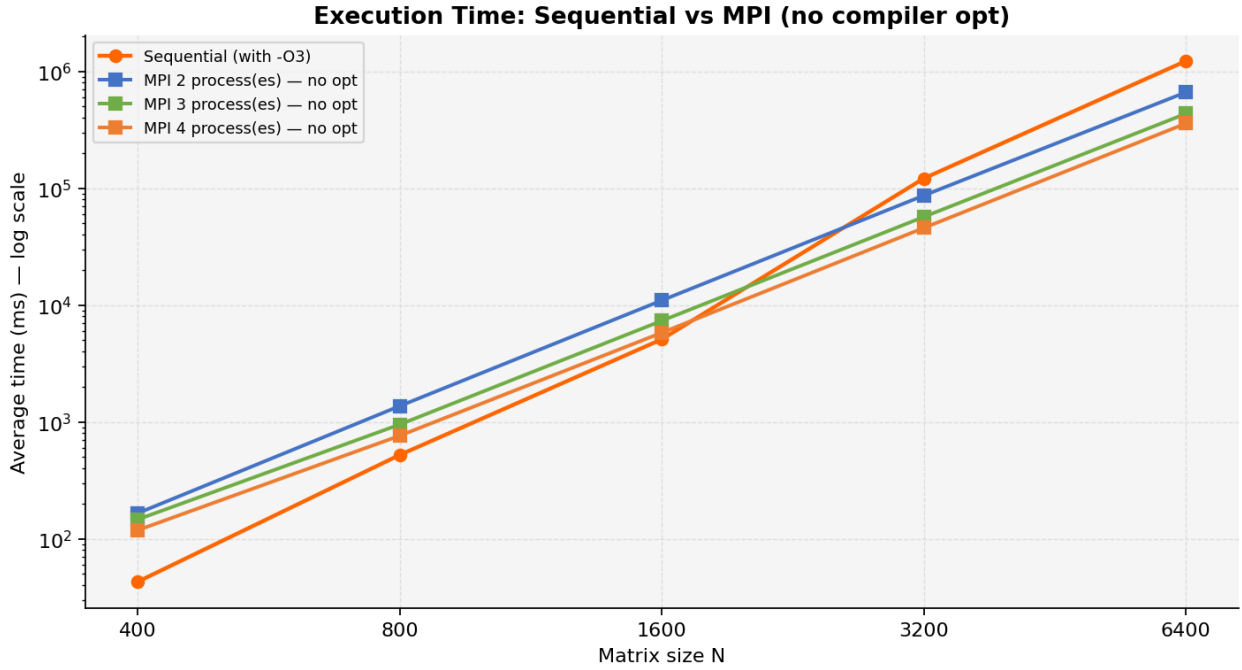


Figura 1: Tiempo de ejecución de la implementación con MPI

4.2.1. Speedup

Table 2 — Average Execution Time and Speedup										
Implementation	N=400 Avg (ms)	N=400 Speedup	N=800 Avg (ms)	N=800 Speedup	N=1,600 Avg (ms)	N=1,600 Speedup	N=3,200 Avg (ms)	N=3,200 Speedup	N=6,400 Avg (ms)	Avg Speedup
Sequential — with -O3 (reference)	42,856	1,00	521,456	1,00	5,090,714	1,00	121.258,014	1,00	1.225.101,240	1,00
MPI 2 process(es) — no compiler optimization	165,380	0,2591	1.366,995	0,3815	10.953,477	0,4648	86.579,105	1,4005	661.609,650	0,87
MPI 3 process(es) — no compiler optimization	146,240	0,2931	950,171	0,5488	7.333,502	0,6942	56.843,394	2,1332	433.896,545	1,30
MPI 4 process(es) — no compiler optimization	118,319	0,3622	761,743	0,6846	5.797,029	0,8782	45.700,831	2,6533	357.920,021	1,60

Cuadro 5: Tabla del speedup de la implementación con MPI sin optimizaciones

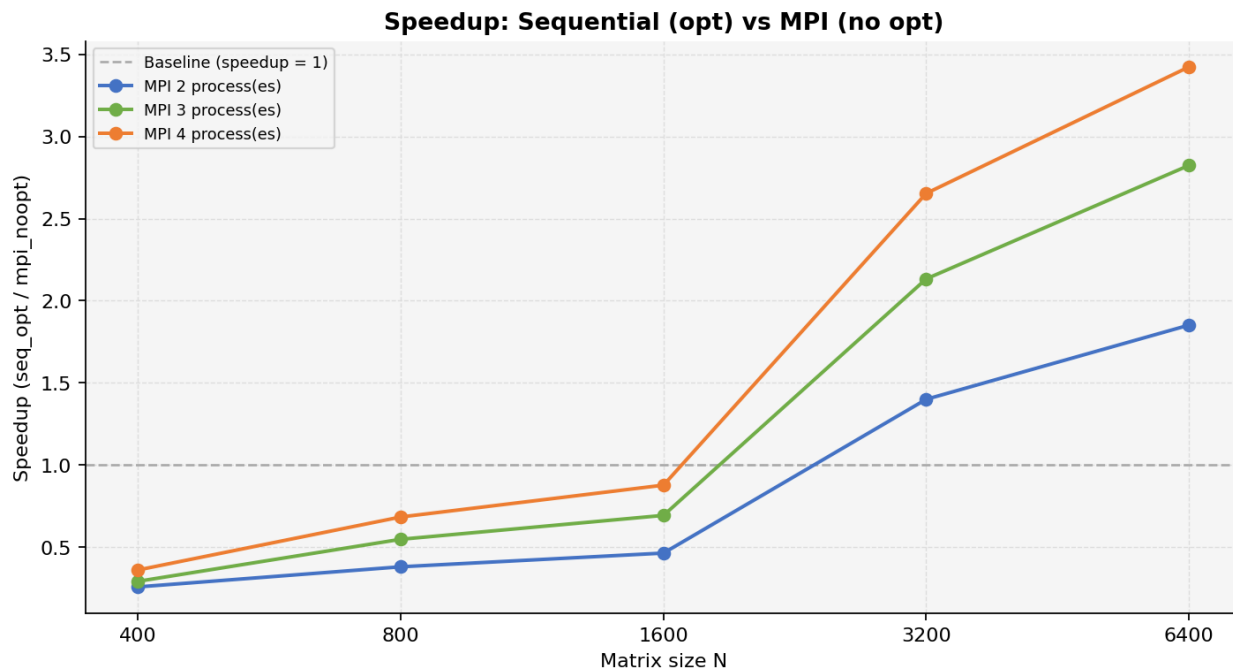


Figura 2: Speedup de la implementación con MPI sin optimizaciones

4.3. Ejecución de la implementación con MPI con optimización por compilador

MPI 2 process(es) — with -O3 -march=native full optimization					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	18,254	95,044	622,153	7.077,018	79.390,431
2	8,157	70,911	562,057	6.496,313	78.693,339
3	8,758	68,814	566,932	5.544,022	77.611,260
4	8,443	69,628	521,283	6.920,359	77.245,611
5	8,048	67,687	561,876	6.664,448	77.818,466
6	8,360	71,752	551,666	6.670,756	77.494,819
7	7,971	71,243	559,927	7.169,774	77.401,568
8	8,326	74,412	588,683	7.605,743	79.086,863
9	7,594	69,785	536,636	6.351,925	79.213,832
10	8,009	69,349	534,199	6.107,716	77.439,786
Average	9,192	72,862	560,541	6.660,807	78.139,598
Std Dev	3,035	7,600	27,420	553,918	809,577
CV (%)	33,02	10,43	4,89	8,32	1,04

Cuadro 6: Tiempo de ejecución de la implementación con MPI con optimizaciones con 2 procesos

MPI 3 process(es) — with -O3 -march=native full optimization					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	40,399	93,936	411,567	4.241,149	53.960,123
2	35,017	81,603	411,073	5.024,263	51.908,641
3	32,096	80,868	420,525	2.790,028	52.055,823
4	34,010	84,893	428,333	4.622,287	52.399,243
5	34,072	82,417	436,260	4.471,480	52.591,219
6	31,671	82,222	396,662	4.104,012	53.341,943
7	32,263	82,668	423,866	4.256,295	52.469,366
8	32,701	82,056	418,286	4.635,842	50.556,542
9	33,184	82,174	420,654	5.102,937	47.494,106
10	32,790	86,787	418,775	4.873,678	52.385,009
Average	33,820	83,962	418,600	4.412,197	51.916,202
Std Dev	2,399	3,707	10,148	627,882	1.698,477
CV (%)	7,09	4,41	2,42	14,23	3,27

Cuadro 7: Tiempo de ejecución de la implementación con MPI con optimizaciones con 3 procesos

MPI 4 process(es) — with -O3 -march=native full optimization					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	31,662	75,605	367,729	4.580,763	41.687,866
2	31,542	74,699	364,453	4.618,981	42.105,482
3	31,921	74,679	364,268	4.605,554	41.561,796
4	31,503	77,893	363,156	4.568,941	41.447,840
5	31,653	77,719	367,965	4.674,471	42.094,323
6	31,455	78,259	374,787	4.673,539	41.976,462
7	31,736	76,313	368,049	4.726,130	41.285,134
8	31,600	79,820	365,508	4.502,189	41.590,374
9	31,495	77,537	364,129	4.571,950	41.587,034
10	31,906	75,866	367,327	4.807,967	41.203,024
Average	31,647	76,839	366,737	4.633,048	41.653,933
Std Dev	0,156	1,590	3,191	84,422	300,304
CV (%)	0,49	2,07	0,87	1,82	0,72

Cuadro 8: Tiempo de ejecución de la implementación con MPI con optimizaciones con 4 procesos

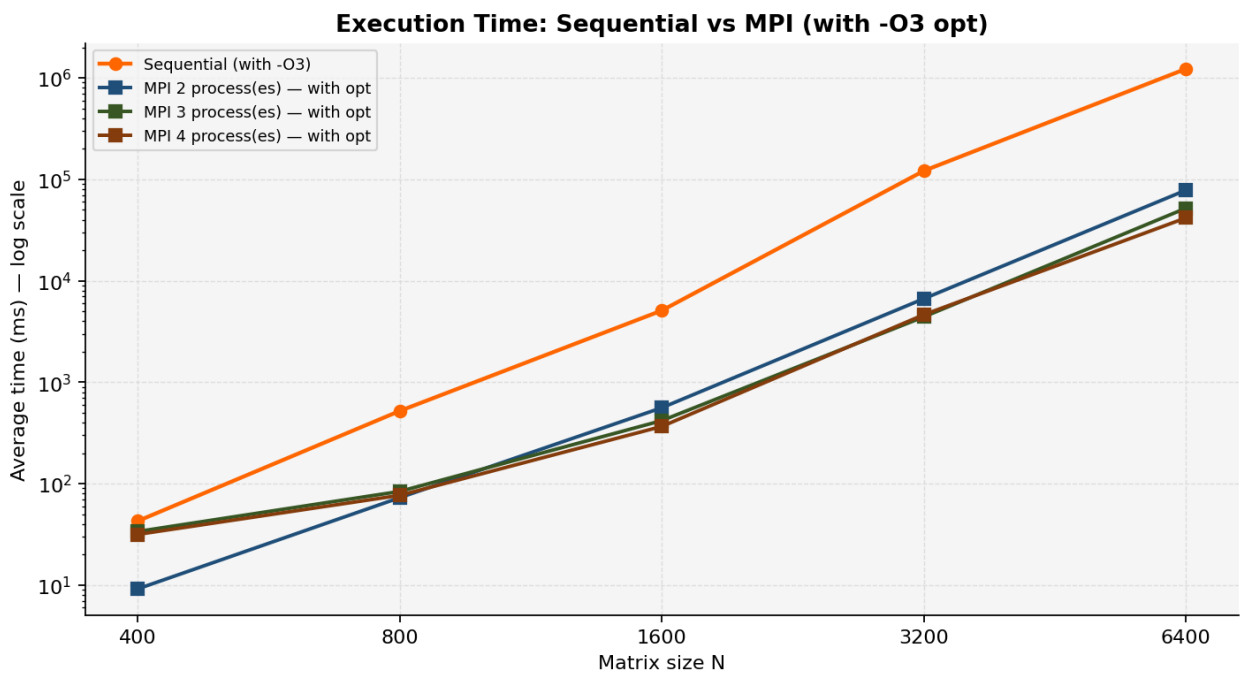


Figura 3: Tiempo de ejecución de la implementación con MPI con optimizaciones

4.3.1. Speedup

Implementation	N=400 Avg (ms)	N=400 Speedup	N=800 Avg (ms)	N=800 Speedup	N=1,600 Avg (ms)	N=1,600 Speedup	N=3,200 Avg (ms)	N=3,200 Speedup	N=6,400 Avg (ms)	Avg Speedup
Sequential — with -O3 (reference)	42,856	1,00	521,456	1,00	5,090,714	1,00	121.258,014	1,00	1.225.101,240	1,00
MPI 2 process(es) — with -O3 -march=native full	9,192	4,6623	72,862	7,1567	560,541	9,0818	6.660,807	18,2047	78.139,598	10,96
MPI 3 process(es) — with -O3 -march=native full	33,820	1,2672	83,962	6,2106	418,600	12,1613	4.412,197	27,4825	51.916,202	14,14
MPI 4 process(es) — with -O3 -march=native full	31,647	1,3542	76,839	6,7863	366,737	13,8811	4.633,048	26,1724	41.653,933	15,52

Cuadro 9: Tabla del speedup de la implementación con MPI con optimizaciones

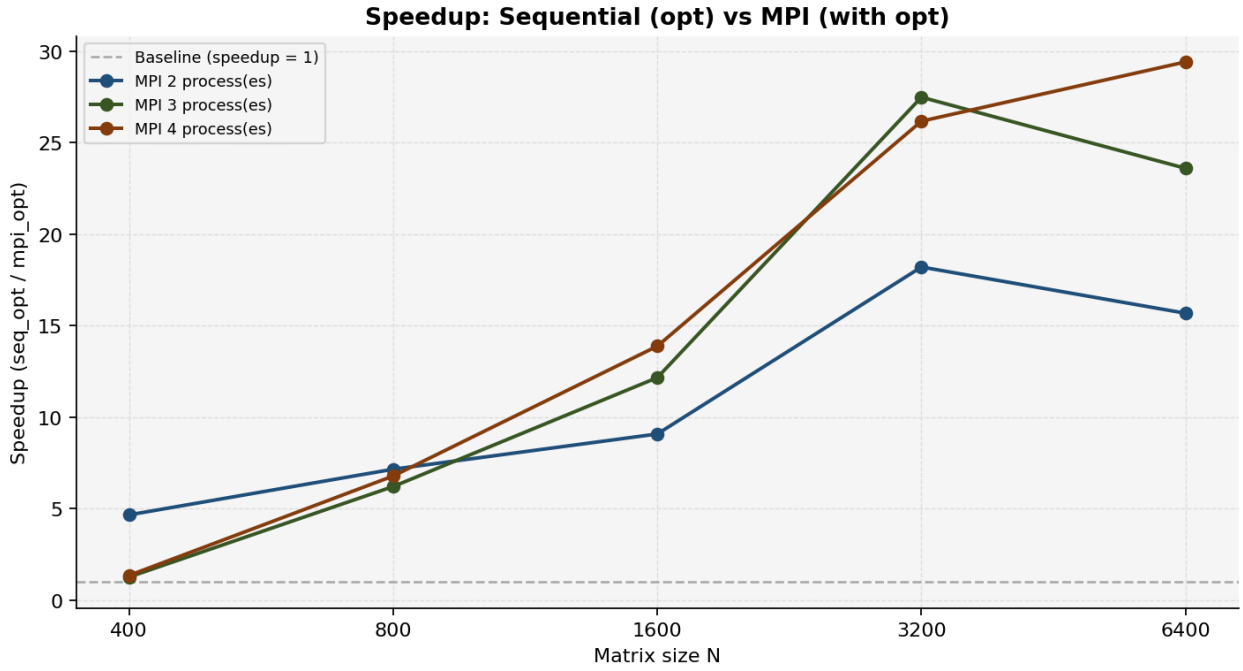


Figura 4: Speedup de la implementación con MPI con optimizaciones

5. Conclusiones

1. Los resultados muestran que la versión MPI sin optimizaciones del compilador obtiene mejoras bastante limitadas frente a la versión secuencial optimizada. El speedup promedio apenas alcanza valores de 0.87 con 2 procesos, 1.3 con 3 procesos y 1.6 con 4 procesos, lo que evidencia que gran parte del tiempo ganado por paralelismo termina siendo consumido por la sobrecarga asociada a la comunicación, sincronización y distribución de datos entre procesos. En este escenario, la paralelización por sí sola no es suficiente para convertir la aplicación en una solución realmente eficiente con respecto a la optimización por compilador usada en el algoritmo secuencial.

2. Cuando se habilitan optimizaciones del compilador, el comportamiento cambia de manera significativa. Los speedups promedio aumentan hasta 10.96 con 2 procesos, 14.14 con 3 procesos y 15.52 con 4 procesos, alcanzando incluso un máximo de 29.41x para el caso de $N = 6400$ con 4 procesos. Esto demuestra que las optimizaciones relacionadas con localidad de memoria, mejor uso de caché, vectorización y reorganización de bucles tienen un impacto decisivo en algoritmos intensivos en memoria como la multiplicación de matrices densas.
3. Se observa además que el rendimiento mejora a medida que aumenta el tamaño del problema. Para matrices pequeñas, los costos de creación de procesos, sincronización y transferencia de información representan una parte importante del tiempo total de ejecución, generando speedups modestos e incluso inferiores a 1 en algunos casos. Sin embargo, conforme el tamaño de la matriz crece, cada proceso realiza una mayor cantidad de trabajo computacional y el costo fijo de coordinación se amortiza, permitiendo que el paralelismo sea mucho más efectivo en tamaños como $N = 3200$ y $N = 6400$.
4. Aunque el rendimiento aumenta al utilizar más procesos, la escalabilidad no es perfectamente lineal. Pasar de 2 a 4 procesos mejora el tiempo total de ejecución, pero el rendimiento no se duplica proporcionalmente. Esto sugiere la existencia de limitaciones asociadas al acceso a memoria, la comunicación MPI y el balance de carga entre procesos. Además, el tiempo total medido incluye operaciones adicionales como lectura desde NFS, distribución mediante `MPI_Scatterv`, recolección con `MPI_Gatherv` y escritura final de resultados, factores que también afectan la eficiencia global.
5. Desde el punto de vista computacional, los resultados son coherentes con la complejidad $O(n^3)$ del algoritmo de multiplicación de matrices. A medida que n aumenta, el costo del cálculo crece mucho más rápido que la sobrecarga de coordinación entre procesos, favoreciendo así el aprovechamiento del paralelismo. Las optimizaciones del compilador reducen considerablemente el costo del kernel principal, permitiendo que cada proceso ejecute una mayor cantidad de trabajo útil y logrando un mejor aprovechamiento del hardware disponible.
6. Finalmente, la versión sin optimizaciones presenta una variabilidad considerablemente mayor en los tiempos de ejecución, especialmente en tamaños pequeños y en configuraciones con pocos procesos. Esto indica que los costos fijos del sistema y el overhead de MPI tienen un peso importante cuando la carga computacional todavía es reducida. En contraste, la versión optimizada muestra un comportamiento mucho más estable y consistente, reflejando una utilización más eficiente de los recursos de cómputo y

memoria.

7. Una de las conclusiones más importantes es que aumentar la cantidad de procesos no garantiza automáticamente una mejora en el rendimiento. Los resultados muestran que, cuando el núcleo de multiplicación no está optimizado, gran parte del tiempo se pierde en coordinación entre procesos, sincronización y acceso a memoria. En otras palabras, para que MPI realmente aporte beneficios, primero es necesario que cada proceso tenga suficiente trabajo útil y que el código base sea eficiente.
8. Los experimentos también evidencian que el principal cuello de botella del problema no está en la capacidad de cálculo del procesador, sino en el acceso a memoria y el manejo de caché. La gran diferencia de rendimiento entre la versión optimizada y la no optimizada demuestra que optimizaciones como el mejor orden de los bucles y el aprovechamiento de la localidad de memoria tienen un impacto mucho mayor de lo esperado. Esto confirma que, en multiplicación de matrices densas, un código bien optimizado puede ser más importante que simplemente agregar más procesos.
9. En términos de escalabilidad, la implementación presenta un comportamiento favorable conforme aumenta el tamaño de la matriz. Para tamaños pequeños, el overhead de MPI y las operaciones de coordinación representan una parte importante del tiempo total, por lo que las mejoras son limitadas. Sin embargo, en matrices grandes como $N = 3200$ y $N = 6400$, el costo del cálculo domina sobre la comunicación y el sistema logra aprovechar mejor el paralelismo disponible, especialmente en la versión optimizada.

Referencias

- Ben-Ari, M. (2006). *Principles of Concurrent and Distributed Programming* (2nd). Addison-Wesley.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd). The MIT Press.
- Dongarra, J. J., Du Croz, J., Duff, I. S., & Hammarling, S. (1990). A Set of Level 3 Basic Linear Algebra Subprograms. *ACM Transactions on Mathematical Software*, 16(1), 1-17. <https://doi.org/10.1145/77626.79170>
- Drepper, U. (2007). *What Every Programmer Should Know About Memory* (Technical Report). Red Hat, Inc. <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
- Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003a). *Introduction to Parallel Computing* (2nd). Addison-Wesley.
- Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003b). *Introduction to Parallel Computing* (2.^a ed.). Addison-Wesley.
- Gropp, W., Lusk, E., & Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (3.^a ed.). MIT Press.
- Hager, G., & Wellein, G. (2010a). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
- Hager, G., & Wellein, G. (2010b). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
- IBM. (2024). ¿Qué es la computación paralela? [Accessed 2026-05-26].
- Lam, M. S., Rothberg, E. E., & Wolf, M. E. (1991). The Cache Performance and Optimizations of Blocked Algorithms. *ACM SIGARCH Computer Architecture News*, 19(2), 63-74. <https://doi.org/10.1145/115953.115963>
- MPI Forum. (2021). MPI: A Message-Passing Interface Standard, Version 4.0 [Accessed 2026-05-26].
- Pacheco, P. S. (1997). *Parallel Programming with MPI*. Morgan Kaufmann.
- Patterson, D. A., & Hennessy, J. L. (2017a). *Computer Architecture: A Quantitative Approach* (6th). Morgan Kaufmann.
- Patterson, D. A., & Hennessy, J. L. (2017b). *Computer Architecture: A Quantitative Approach* (6.^a ed.). Morgan Kaufmann.
- Sandberg, R., Goldberg, D., Kleiman, S., Walsh, D., & Lyon, B. (1985). Design and Implementation of the Sun Network Filesystem. *USENIX Conference Proceedings*.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th). Wiley.

- Snir, M., Otto, S., Huss-Lederman, S., Walker, D., & Dongarra, J. (1998). *MPI: The Complete Reference, Volume 1: The MPI Core* (2.^a ed.). The MIT Press.
- Stone, H., & Hennessy, J. L. (1995). Performance of Network File Systems. *IEEE Transactions on Computers*.
- Stoner, J. C., Terry, M., et al. (1995). The Design and Implementation of a Network File System. *ACM Operating Systems Review*.
- Storm, R. L., et al. (1988). NFS and the Distributed File System Paradigm. *IEEE Computer*.
- Strang, G. (2016). *Introduction to Linear Algebra* (5th). Wellesley-Cambridge Press.
- Tanenbaum, A. S., & Steen, M. V. (2015). *Distributed Systems: Principles and Paradigms* (2.^a ed.). Pearson.
- Terry, D. B. (1995). Session Guarantees for Weakly Consistent Replicated Data. *ACM SIGOPS Operating Systems Review*.
- Yi, L., Li, C., & Guo, J. (2020). CPI for Runtime Performance Measurement: The Good, the Bad, and the Ugly. *IEEE International Symposium on Workload Characterization (IISWC 2020)*, 15-25. <https://doi.org/10.1109/IISWC50251.2020.00011>